

Actividad 09

Laboratorio: File Path Traversal, Traversal Sequences Stripped Non-Recursively

PortSwigger Web Security Academy

Elaborado por: Juan Pablo Casanova De La Madrid

Herramienta utilizada: Burp Suite Community Edition

1. Nombre del laboratorio

File path traversal, traversal sequences stripped non-recursively

Laboratorio perteneciente a PortSwigger Web Security Academy, dentro de la categoría File path traversal.

2. Nivel

Practitioner.

Al igual que en el laboratorio de bypass con ruta absoluta, este ejercicio incorpora una mitigación en la aplicación: se eliminan las secuencias de traversal ("..") del valor recibido antes de utilizarlo. Sin embargo, dicha eliminación se realiza de forma no recursiva, es decir, se ejecuta una sola vez sobre la cadena y no se vuelve a aplicar sobre el resultado, lo que abre la puerta a una técnica de bypass distinta a la usada en el laboratorio anterior.

3. Tipo de explotación

Path Traversal / Directory Traversal con bypass de filtro no recursivo (CWE-22: Improper Limitation of a Pathname to a Restricted Directory).

Es una vulnerabilidad del lado del servidor que permite a un atacante manipular la ruta de un archivo solicitado para acceder a archivos fuera del directorio raíz previsto por la aplicación. En este caso, la explotación requiere evadir un filtro que elimina las secuencias ".." del valor de entrada una única vez, construyendo un payload que, tras esa única eliminación, se reconstruye por sí mismo en una secuencia de traversal válida.

4. Objetivo del laboratorio

El laboratorio contiene una vulnerabilidad de path traversal en la funcionalidad que muestra las imágenes de los productos de la tienda. La aplicación elimina las secuencias de traversal del nombre de archivo proporcionado por el usuario antes de utilizarlo, pero lo hace de forma no recursiva.

Objetivo: explotar dicha vulnerabilidad, evadiendo el filtro de eliminación no recursiva mediante secuencias de traversal anidadas, para recuperar el contenido del archivo del sistema /etc/passwd, lo que confirma la lectura arbitraria de archivos y da por resuelto el laboratorio.

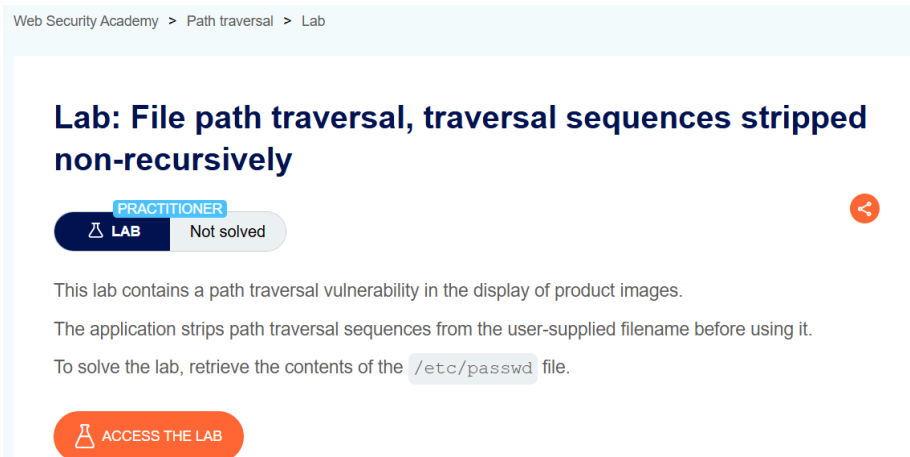


Figura 1. Descripción y objetivo del laboratorio en PortSwigger Web Security Academy (nivel Practitioner, estado Not solved).

5. Explicación técnica de la vulnerabilidad

La aplicación web muestra el catálogo de productos ("We Like To Shop") y cada imagen se sirve dinámicamente mediante un endpoint del tipo:

GET /image?filename=<nombre_de_archivo>

El parámetro filename es utilizado por el servidor para construir, de forma interna, la ruta física del archivo a devolver. Como mecanismo de defensa, la aplicación busca y elimina del valor recibido cualquier ocurrencia de la secuencia "../" antes de usarlo para acceder al sistema de archivos.

El problema es que este filtrado se aplica de forma no recursiva: la aplicación recorre la cadena una sola vez, eliminando cada "../" que encuentra, pero no vuelve a examinar el resultado por sí, tras esa eliminación, se ha formado una nueva secuencia de traversal. Esto permite construir un payload anidado como "....//", en el que, al eliminar internamente el patrón "../" contenido en el medio de la cadena, el resultado que queda es exactamente "../".

6. Procedimiento paso a paso

1. Se inicia Burp Suite Community Edition, seleccionando "Temporary project in memory" y la configuración por defecto ("Use Burp defaults").

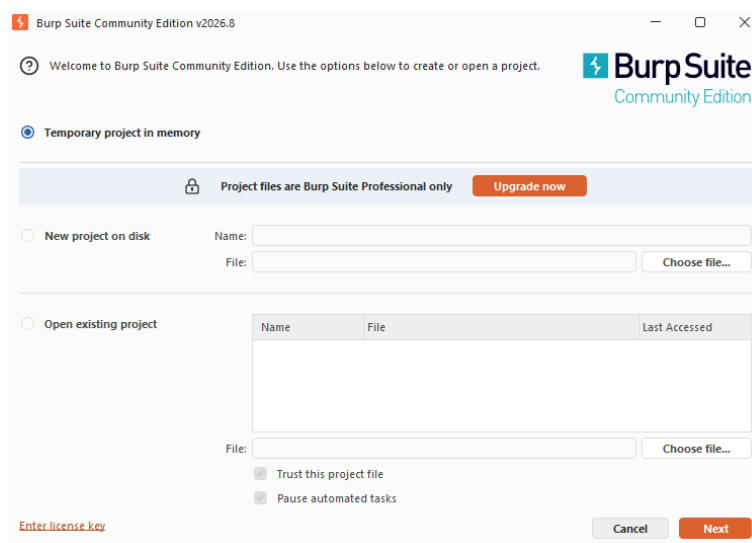


Figura 2. Creación de un proyecto temporal en memoria.

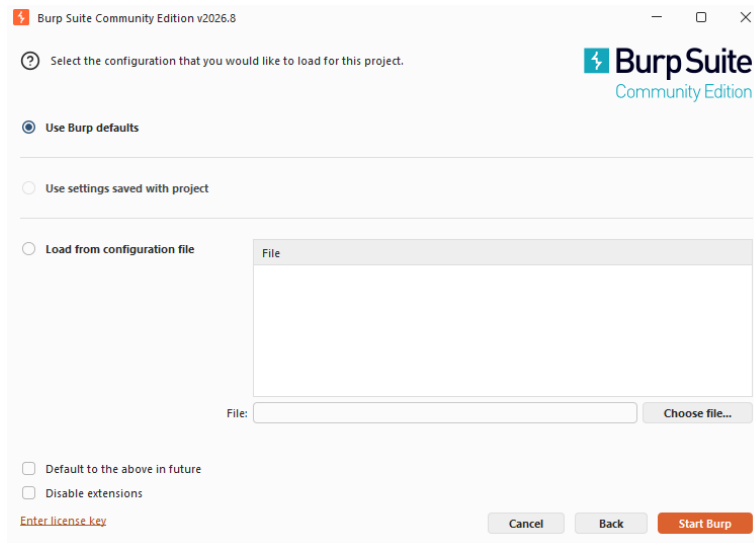


Figura 3. Selección de la configuración por defecto de Burp.

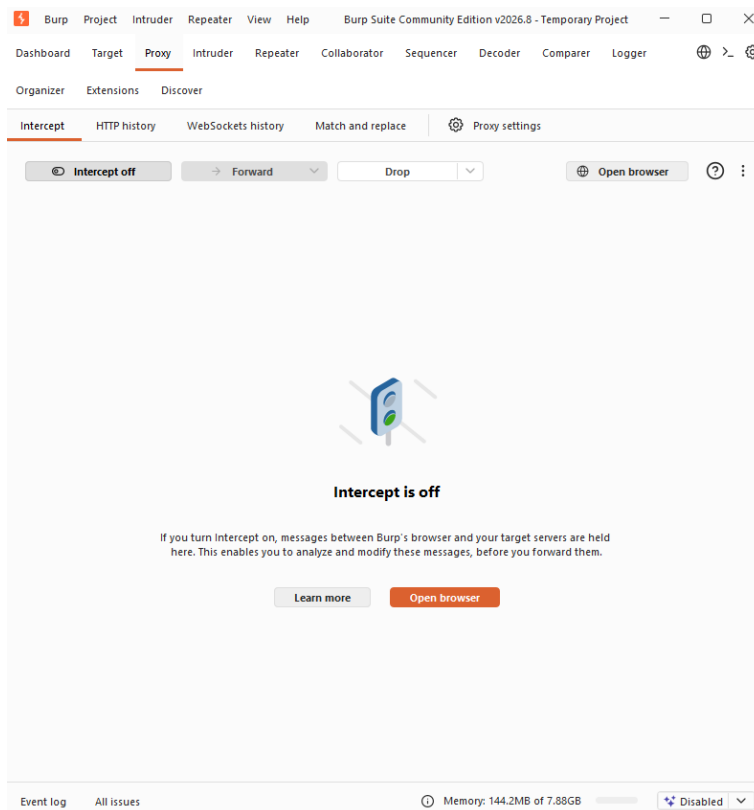


Figura 4. Burp Suite iniciado, pestaña Proxy con Intercept inicialmente apagado.

2. Se accede al laboratorio desde la Web Security Academy de PortSwigger, con el tráfico pasando a través del navegador integrado de Burp.

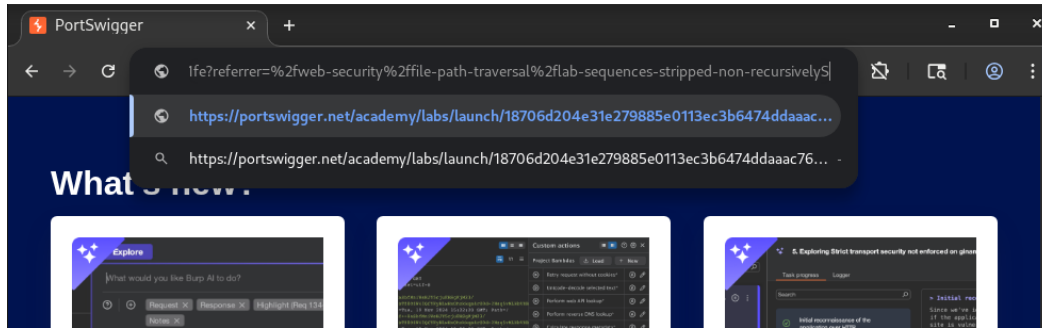


Figura 5. Navegación hacia la URL del laboratorio a través del navegador de Burp.

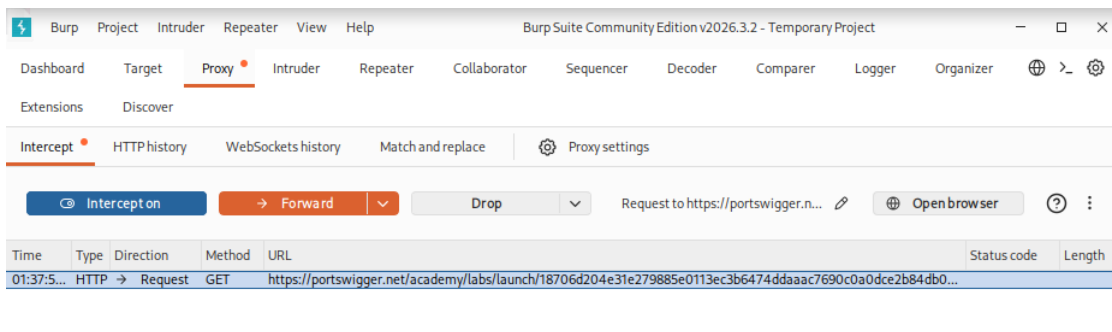


Figura 6. Petición GET inicial capturada en el historial del Proxy al abrir el laboratorio.

3. Con el Intercept activo, se navega por la tienda; Burp captura las peticiones GET que la página realiza para cargar cada imagen de producto, del tipo `/image?filename=NN.jpg`, así como la vista general del catálogo.

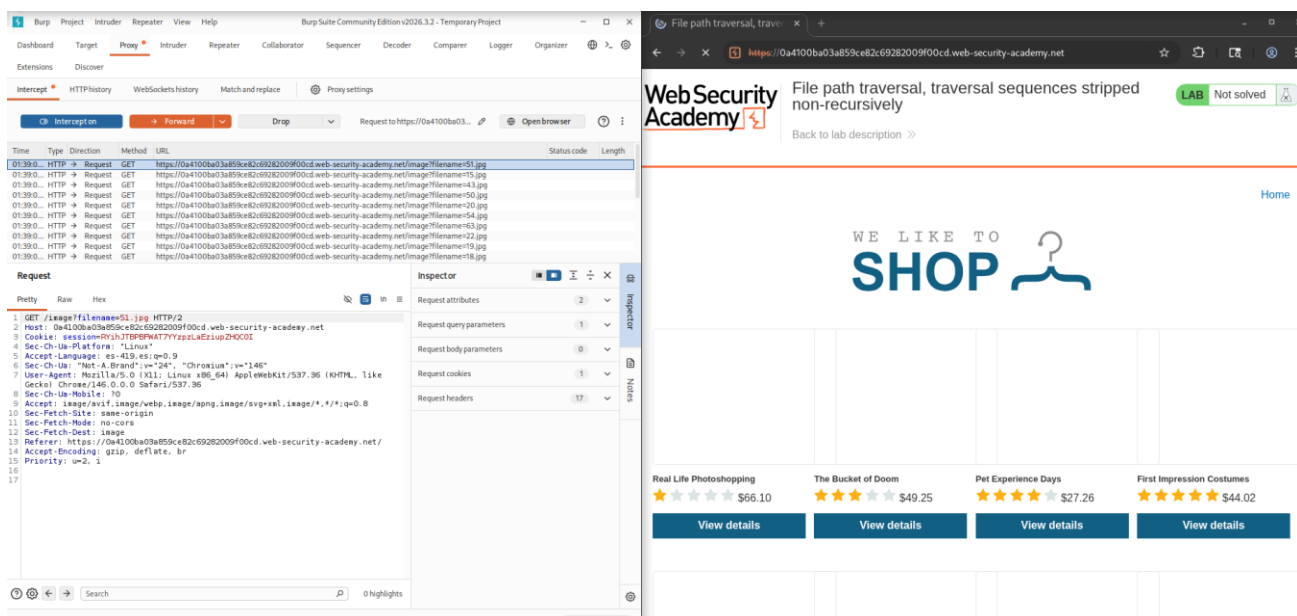


Figura 7. Petición interceptada GET /image?filename=51.jpg junto con el resto de las peticiones de imágenes en el historial del Proxy.

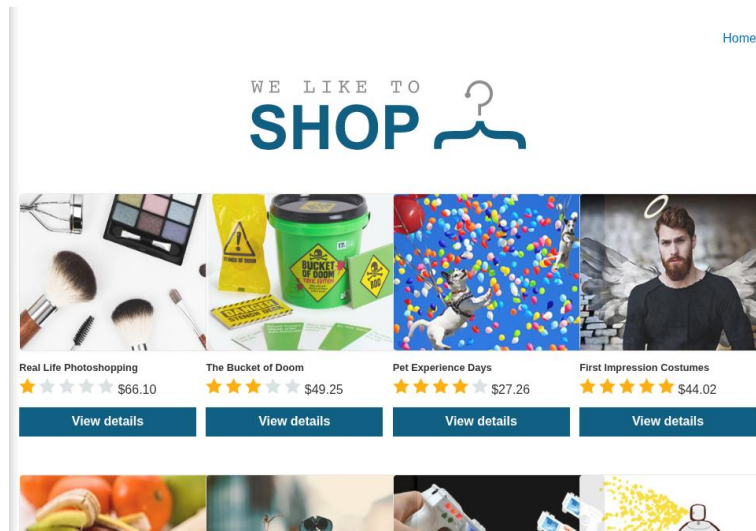


Figura 8. Vista de la tienda mostrando los productos cuyas imágenes se cargan mediante el parámetro filename.

- Se da clic en "View details" de uno de los productos; Burp intercepta la petición GET /product?productId=1 correspondiente, y se reenvía con "Forward" para poder acceder a la página del producto.

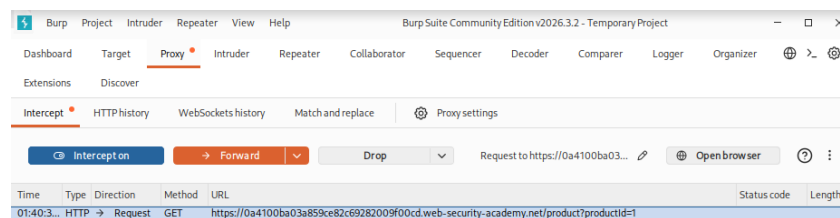


Figura 9. Petición GET /product?productId=1 interceptada tras dar clic en "View details".

- Una vez capturada la petición GET /image?filename=51.jpg correspondiente a la imagen del producto, se envía al módulo Repeater ("Send to Repeater") para poder modificarla y reenviarla libremente.

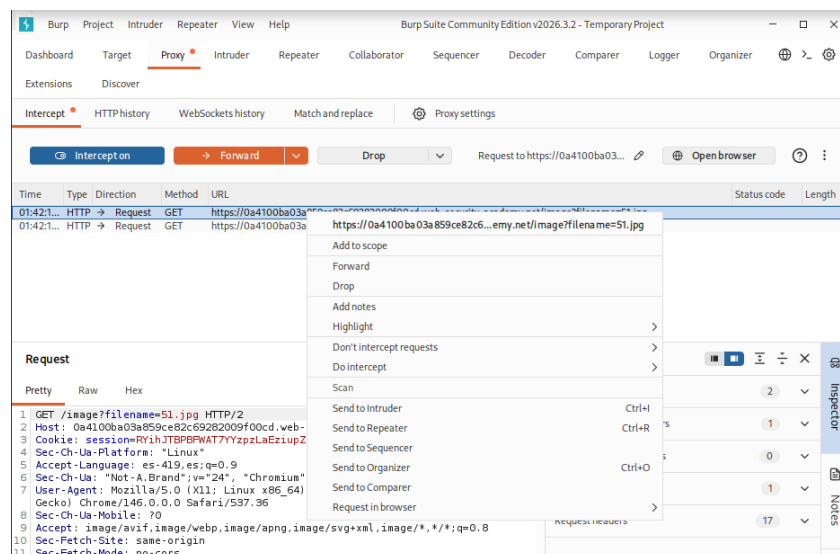


Figura 10. Menú contextual: envío de la petición GET /image?filename=51.jpg a Repeater.

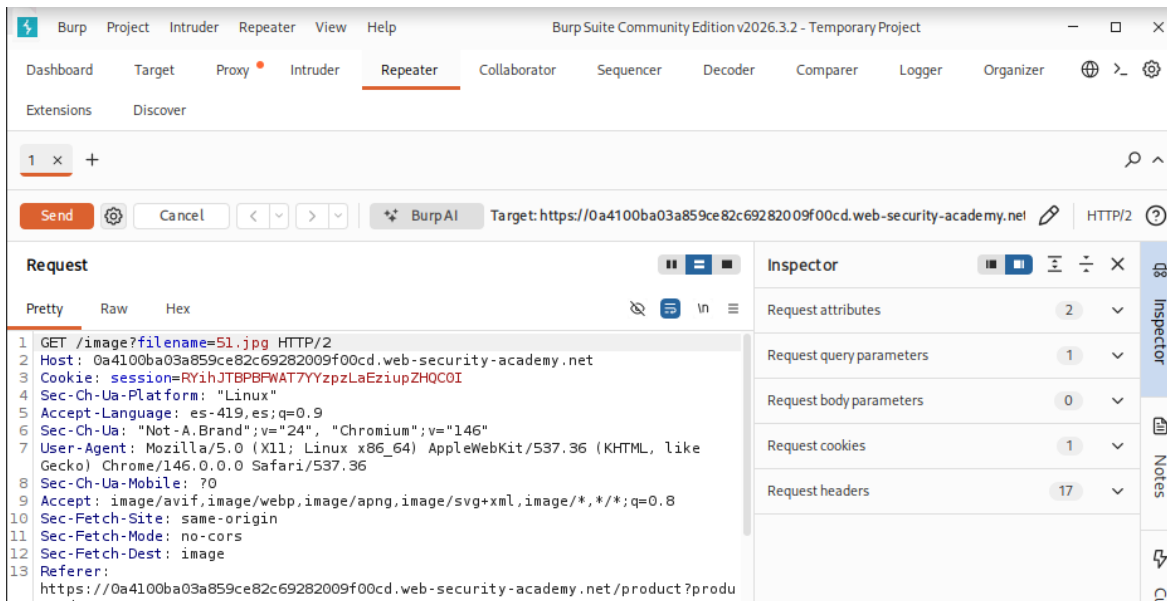


Figura 11. Petición original en Repeater antes de modificar el payload (filename=51.jpg).

- En Repeater, se reemplaza el valor 51.jpg por la secuencia de traversal anidada `....//....//....//etc/passwd`, diseñada para evadir el filtrado no recursivo de la aplicación.

GET /image?filename=....//....//....//etc/passwd HTTP/2

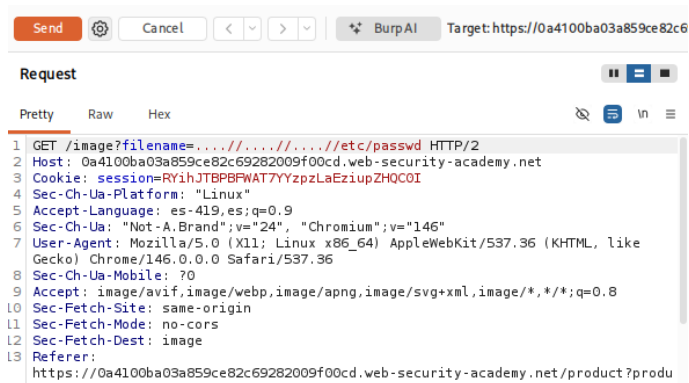


Figura 12. Payload de traversal anidado inyectado en el parámetro filename.

- Se hace clic en Send y se analiza la respuesta del servidor.

8. El servidor responde con código HTTP/2 200 OK y devuelve en el cuerpo de la respuesta el contenido íntegro del archivo `/etc/passwd`, confirmando la explotación exitosa de la vulnerabilidad.

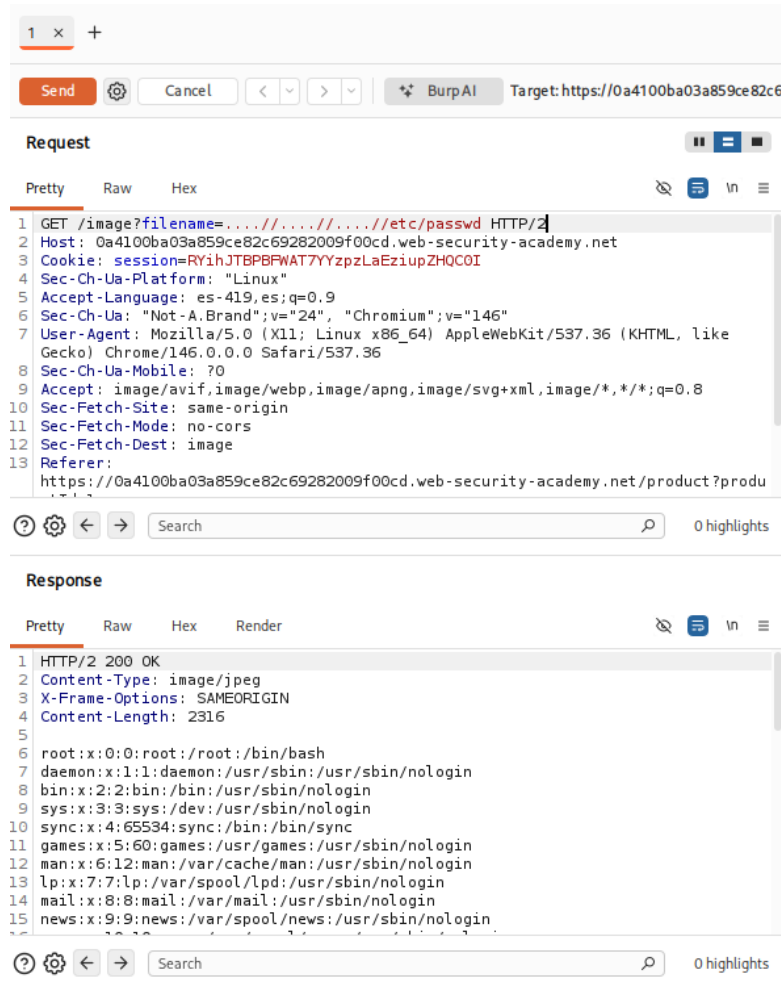


Figura 13. Petición con el payload y respuesta vulnerable (200 OK) mostrando el contenido de `/etc/passwd`.

9. Finalmente, se regresa a la página del laboratorio en PortSwigger, la cual actualiza automáticamente su estado a "Solved", confirmando que el objetivo de la actividad fue completado.

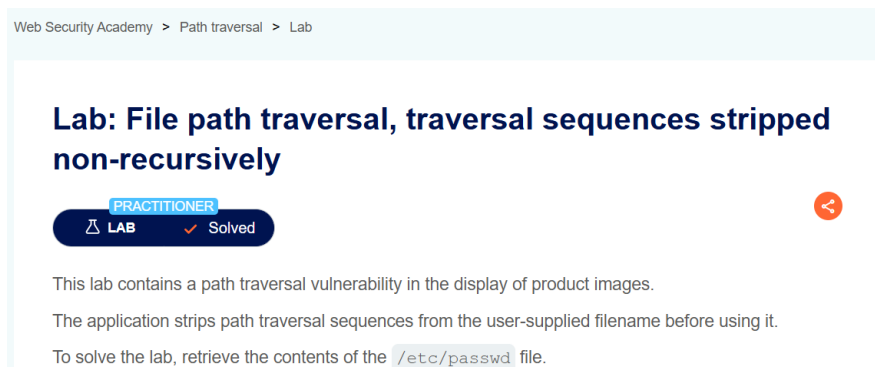


Figura 14. Confirmación de laboratorio resuelto ("Solved").

7. Explicación del payload

GET /image?filename=....//....//....//etc/passwd HTTP/2

El payload se construye aprovechando específicamente la forma en que el filtro elimina las secuencias de traversal:

- **....//** : cada bloque está formado por cuatro puntos seguidos de una doble diagonal. Si la aplicación busca y elimina la subcadena "../" dentro de este bloque, lo que encuentra es el patrón ../ ubicado entre el segundo y el cuarto carácter (".." + "." + "/" reorganizados como "....//". Este contiene "../" en su interior). Al eliminarlo una sola vez, los caracteres restantes se recomponen exactamente en "../", ya que el filtro no vuelve a analizar el resultado tras la sustitución.
- **Repetición del bloque:** al encadenar tres veces "....//", tras el filtrado no recursivo se obtiene el equivalente a "../..../", suficiente para retroceder desde el directorio de imágenes hasta la raíz del sistema de archivos.
- **/etc/passwd:** una vez en la raíz, se referencia este archivo estándar de Unix/Linux, legible por cualquier usuario del sistema, cuyo contenido confirma de forma clara e inequívoca la lectura arbitraria de archivos y permite resolver el laboratorio.

En síntesis, el payload no intenta ocultar la secuencia de traversal, sino que explota la falta de recursividad del filtro: la propia acción de "limpiar" la entrada es la que genera la secuencia maliciosa final.

8. Mitigación recomendada

Para prevenir vulnerabilidades de path traversal como la explotada en este laboratorio, se recomienda aplicar las siguientes medidas, de forma combinada:

- Aplicar el filtrado de secuencias de traversal de forma recursiva o, preferentemente, en un único paso mediante una expresión regular que capture todas las variantes posibles, en lugar de una sola pasada de eliminación de subcadenas.
- Validar el parámetro filename contra una lista blanca (whitelist) de nombres de archivo o identificadores permitidos, evitando por completo depender de la eliminación de patrones maliciosos.
- Resolver la ruta final de forma canónica (canonicalización) con funciones seguras del lenguaje o framework, y verificar después de resolverla que el resultado siga estando dentro del directorio base permitido, en lugar de sanear la cadena de entrada antes de construir la ruta.
- Evitar exponer directamente el sistema de archivos del servidor a partir de datos proporcionados por el usuario; en su lugar, mapear identificadores de producto/imagen a rutas físicas mediante una tabla o índice controlado por el backend.
- Ejecutar el proceso de la aplicación con el mínimo privilegio necesario, restringiendo los permisos de lectura del usuario del servicio web únicamente a los directorios estrictamente necesarios.
- Reforzar la defensa en profundidad con un WAF (Web Application Firewall) que detecte patrones de traversal anidados y codificados, y aplicar monitoreo/alertas sobre accesos a archivos fuera de lo esperado.